PERINKPHERAL

Платформа приложений
Техническое описание и руководство разработчика

Дисплей	4,2" E-Ink, 400 x 300 пикселей, 1 бит
Ввод	4 кнопки
Приложение	Модуль WebAssembly и манифест
Язык	C, без стандартной библиотеки
Интерфейс	один заголовочный файл, sdk/host_api.h, ABI 2
Память	64 КБ на приложение
Сборка и загрузка	около 10 с, без перепрошивки

I Техническое описание: параметры и модель исполнения

II Руководство разработчика: пакет, экран, ввод, хранение, сеть

III Справочник: API хоста и библиотека SDK

редакция 2, 2026-11-14 / API хоста по sdk/host_api.h, abi 2 / предварительная версия: до первого релиза API хоста не зафиксирован

github.com/GrakovNe/perinkpheral / MIT

Содержание

Часть I. Техническое описание 4

1. Обзор 4
   - Возможности 4
   - Как работает приложение 4
2. Характеристики 5
   - Дисплей 5
   - Ввод 5
   - Песочница 5
   - Хранение 6
   - Сеть 6
   - Время и фоновая работа 6
   - Текст 7
3. Модель исполнения 7
   - Жизненный цикл приложения 7
   - Гарантии для обработчиков 7
   - Состояние 8
   - Навигация и обновление экрана 8

Часть II. Руководство разработчика 9

4. Первое приложение 9
   - Требования 9
   - Создание, сборка, запуск 9
   - Пример приложения 10
   - Отчёт сборки 10
5. Пакет 11
   - Манифест 12
   - Опросы 13
   - Настройки 13
6. Экран 14
   - Поверхность и цвет 14
   - Шрифты 14
   - Текст за пределами ASCII 14
   - Картинки и иконки 15
   - Kit 16
   - Запрос обновления экрана 17
7. Кнопки 17
   - BACK 18
   - Списки, выбор значения, даты 18
8. Хранение 18
   - Хранилище 18
   - Файлы 19
9. Сеть 19
   - Значения, извлечённые хостом 19
   - Потоковые ответы 19
   - Запросы из кода 20
   - Неудавшиеся запросы 20
10. Фоновая работа 21

Уведомления 21
Будильники, таймеры 21
11. Время 21
12. Языки 22
13. Правила проектирования 22
Содержимое экрана 22
Память 23
Состояние и время 23
Взаимодействие и данные 23
14. Инструменты и отладка 23
Типичные проблемы 24
15. Публикация 24
Часть III. Справочник 26
16. Коды возврата 26
17. Экспорты 26
18. Отрисовка 27
19. Kit 27
20. Навигация, хранение и сеть 28
21. Время 29
22. Системный API 30
23. Библиотека SDK 30
app_text.h: короткие строки без printf 30
app_draw.h, app_state.h: строки, состояние, настройки 31
app_values.h: значения, извлечённые хостом 31
app_ui.h: обработка кнопок на экранах Kit 31
app_alloc.h, json_pick.h и генерируемые файлы 31
24. Лимиты 32

Часть I. Техническое описание

1 Обзор

perINKpheral - настольный дисплей: чёрно-белая панель E-Ink 4,2" с четырьмя кнопками на базе ESP32-C6. Отображается информация, рассчитанная на чтение с одного взгляда: время, погода, ближайшая встреча, состояние дома.

Прошивка называется *хостом*. Меню и настройки предоставляются хостом. Все остальные экраны предоставляются *приложениями*. Приложения устанавливаются из каталога; прошивка при этом не перезаписывается.

Приложение состоит из модуля WebAssembly и манифеста. Модуль пишется на C с одним заголовочным файлом, `sdk/host_api.h`, и исполняется интерпретатором (wasm3) внутри хоста.

Собственные страницы хоста, Settings и About, собраны как приложения с тем же заголовочным файлом.

Возможности

- Дисплей: 400 x 300 пикселей, 1 бит, оттенки серого дизерингом, частичное обновление примерно за 0,5 с
- Ввод: четыре кнопки; события короткого нажатия, длинного нажатия, нажатия и отпускания
- Песочница: 64 КБ собственной памяти; доступ в сеть только через хост
- Сеть: запросы HTTP и HTTPS выполняются хостом - опросы по расписанию или запросы из кода
- Данные: значения из ответов JSON извлекаются хостом; любой ответ может передаваться потоком, фрагментами по 1 КБ
- Фоновая работа: ответы, будильники и события устройства доставляются приложению, когда оно не на экране
- Хранение: хранилище «ключ-значение» и файлы, сохраняются при перезагрузке и переустановке
- Дизайн-система: списки, сообщения, выбор значения, ячейки и строка кнопок, отрисовываются хостом
- Уведомления: показываются хостом поверх любого экрана
- Настройки: объявляются в манифесте, редактируются на веб-странице устройства
- Локализация: строки по языкам, загружаются хостом; латиница с диакритикой, кириллица и греческий
- Инструменты: сборка, проверка и измерение приложения; управление устройством по сети

Как работает приложение

Приложение - это набор экспортируемых обработчиков с фиксированными именами. Обработчик вызывается хостом при наступлении события: нажатие кнопки, таймер, поступление данных, запрос на отрисовку.

Состояние приложения изменяется в обработчике, после чего запрашивается перерисовка. `app_on_draw()` вызывается хостом, когда нужен кадр. Момент и способ обновления экрана определяются хостом.

Одновременно загружено одно приложение. Приложение выгружается, когда уходит с экрана. Постоянное состояние хранится в хранилище приложения; глобальные переменные сохраняются до выгрузки.

2 Характеристики

Дисплей

Параметр	Значение	Примечания
Панель	4,2" E-Ink, SSD1683	Без подсветки. Изображение сохраняется без питания
Разрешение	400 x 300 px	Около 120 пикселей на дюйм
Поверхность приложения	400 x 260 px	Под заголовком хоста. 400 x 300 при `"fullscreen": true`
Глубина цвета	1 бит	17 уровней серого фиксированным дизерингом 4 x 4; для заливок
Частичное обновление	0,46 - 0,53 с	Передаётся только изменившийся прямоугольник
Полное обновление	2,2 с (2,7 с)	Быстрая форма сигнала (форма сигнала по умолчанию). Панель мигает
Интервал полного обновления	каждые 5 частичных обновлений	Настройка пользователя. Убирает остаточное изображение после частичных обновлений
Обновление после нажатия кнопки	немедленно	
Обновление без нажатия кнопки	не чаще 1 раза в 60 с	Настройка пользователя. Производитель панели рекомендует 180 с
Частота последовательности кадров	около 1,5 кадра/с	`HOST_REDRAW_NO_BLINK`. Остаточное изображение накапливается до конца последовательности

Ввод

Параметр	Значение	Примечания
Кнопки	4	Порядок на устройстве: UP, OK, DOWN, BACK
Событий на кнопку	4	Нажата, отпущена, короткое нажатие, длинное нажатие
Длинное нажатие	600 мс	Сообщается, пока кнопка удерживается
Зарезервировано	длинное нажатие BACK	Обрабатывается хостом: очищает стек и открывает меню

Песочница

Параметр	Значение	Примечания
Линейная память	64 КБ, фиксированная	Одна страница wasm: стек 8 КБ, статические данные, остаток для `app_alloc()`. Размер фиксирован
Модуль (`app.wasm`)	до 48 КБ	Исполняется из флеш-памяти
Память интерпретатора	64 КБ	Около 11 КБ базовых плюс 4 байта на каждый байт исполненного кода
Исполняемый код, пройдены все ветви	около 13 КБ	Следует из памяти интерпретатора. `mkapp` предупреждает о превышении оценки
Лимит времени обработчика	3 с	По истечении обработчик останавливается, приложение завершается
Загруженных экземпляров	1	На все приложения

Параметр	Значение	Примечания
Числа с плавающей точкой	только целые	Сборка отклоняет модуль с операцией над числами с плавающей точкой
Библиотека C	`sdk/lib`	Строки, числа, арена-аллокатор, семейство `memcpy`. Стандартной библиотеки C нет
Установленных приложений	6	На устройство. Размер каталога не ограничен

Хранение

Параметр	Значение	Примечания
Ключ хранилища	1 - 15 символов	Символы от `!` до `~`
Значение хранилища	1024 байта	
Хранилище целиком	4096 байт	На приложение. Сохраняется после удаления и доступно после переустановки
Файлы целиком	64 КБ	На приложение: файлы пакета и файлы, записанные кодом. Каждый файл считается целыми блоками по 4 КБ (16 блоков). Гарантируется каждому установленному приложению
Имя файла	1 - 23 символа	`a–z 0–9 . – _`; первый символ - буква, цифра, – или _

Сеть

Параметр	Значение	Примечания
Транспорт	HTTP, HTTPS	Выполняется хостом
Доверие HTTPS	корневые сертификаты браузеров	Самоподписанные сертификаты отклоняются. Для домашних сетей доступен обычный `http`
Адреса для запросов из кода	4	Перечислены в манифесте либо берутся из настройки типа `url` или `link`. Опросы используют адрес из своего `url`
Опросов в манифесте	4	Интервал от 10 с до 1 суток; на батарее не менее 300 с
Одновременных запросов	1	На устройство. Очередь: ещё 2
Заголовки / тело запроса	1023 / 512 байт	До 4 строк заголовков
Ответ с извлечёнными значениями	256 КБ	JSON; до 16 путей; значение до 63 байт
Потоковый ответ	4 МБ	Фрагменты до 1024 байт; около 90 КБ/с по обычному HTTP
Перенаправления	3	Только GET. Запросы из кода остаются в пределах списка адресов
Время запроса HTTPS	около 6 с	В основном TLS-рукопожатие

Время и фоновая работа

Параметр	Значение	Примечания
Таймеры	4	Однократные, в миллисекундах. Действуют, пока приложение на экране
Будильники	4	Относительные, абсолютные или повторяющиеся с периодом от 60 с до 1 суток. Действуют на экране и вне его. Сбрасываются при перезагрузке

Параметр	Значение	Примечания
События устройства (интенты)	3	Запуск, изменение времени, появление сети
Уведомлений в очереди	4	На устройство
Захват экрана	не чаще 1 раза в 30 с	`request_focus()`; разрешается хостом
Часы	секунды Unix	Устанавливаются из сети. 0 до первой установки. Часовой пояс - настройка хоста

Текст

Параметр	Значение	Примечания
Кодировка	UTF-8	Весь текст. Передаётся указателем и длиной
Встроенные глифы	печатные символы ASCII, °	
Пакеты глифов	3	Латиница с диакритикой, кириллица, греческий. Устанавливаются пользователем на веб-странице устройства
Замена	транслитерация в ASCII	"Moskva", "Strasse", "Athina"; иначе ?
Текст на один вызов отрисовки	127 байт	Более длинный текст обрезается по границе символа
Размеры шрифта	6	Высота прописных от 8 до 80 пикселей. Две гарнитуры, выбирает пользователь
Языки	27	Английский встроен; остальные языки - таблицы, загружаемые хостом (раздел 12)

3 Модель исполнения

Приложение загружается для вызова обработчика и может быть выгружено после него.

Жизненный цикл приложения

Событие	Экземпляр	Обработчики, по порядку
Пользователь открывает приложение	Загружается; остаётся, пока владеет экраном	`app_init, app_on_focus, app_on_draw ... app_on_blur, app_on_terminate`
Ответ, будильник, интент или изменение настроек, пока приложение на экране	Тот же экземпляр	`app_on_data / app_on_values / app_on_alarm / app_on_intent / app_on_settings`
То же, пока приложение не на экране	Загружается на время вызова, выгружается после него. Текущий экран приостанавливается и восстанавливается	`app_init, обработчик, app_on_terminate`
`request_focus()` разрешён хостом	Остаётся, на вершине стека	`... app_on_focus, app_on_draw`
Приложение уходит с экрана: BACK, меню, другое приложение сверху	Выгружается	`app_on_blur, app_on_terminate`

Гарантии для обработчиков

- Обработчики выполняются по одному, каждый до завершения. Блокировки не нужны.
- Обработчик ограничен 3 секундами. Обработчик, превысивший лимит или вызвавший ловушку (например, на неверном указателе), останавливается. Приложение завершается, показывается экран ошибки. Хост продолжает работу.
- Обработчики возвращают `void`. Перерисовка, экран или выход получаются запросом. Запросы применяются после возврата из обработчика.

- `app_init()` вызывается раньше любого другого обработчика экземпляра.
- События кнопок доставляются по порядку, без потерь. Ожидающие таймеры с одним номером объединяются. Фрагменты одного ответа доставляются по порядку, последовательными вызовами.
- Фоновая доставка ответа может быть прервана. При нажатии кнопки экземпляр выгружается, оставшиеся фрагменты отбрасываются. Вызывается только `app_on_terminate()`. Опрос повторяется через минуту.

Состояние

Глобальные переменные сохраняются до выгрузки приложения. Выгрузка происходит при каждом уходе приложения с экрана и после каждого фонового вызова.

Состояние загружается в `app_init()` и сохраняется в `app_on_terminate()` - последнем вызове каждого экземпляра. Значение, которое должно пережить отключение питания, сохраняется сразу после изменения.

Метки времени, действительные после перезагрузки: `time_now()` либо `time_uptime_ms()` вместе с `time_boot_id()`. Идентификатор загрузки меняется после каждой перезагрузки.

Навигация и обновление экрана

Навигация организована как стек. Внизу находится меню. Приложение, открытое из меню, помещается сверху. Уведомление, вопрос хоста или приложение, запросившее экран, помещается над текущим экраном.

Верхний элемент снимается коротким нажатием BACK. Длинным нажатием BACK стек очищается и открывается меню.

Загружен только верхний элемент. Приложение ниже в стеке выгружено; его состояние восстанавливается при возврате на вершину.

Панелью управляет хост. Каждый новый кадр сравнивается с кадром на панели, передаётся только изменившаяся область. Обновление без нажатия кнопки откладывается, если предыдущее было слишком недавно.

На панели всегда показан целый кадр. Момент обновления определяется хостом.

Часть II. Руководство разработчика

4 Первое приложение

Требования

- **wasi-sdk** версии 34 или новее. Содержит clang для цели wasm32. Укажите путь к нему в `WASI_SDK_PATH`.
- **Python 3**. Инструменты используют только стандартную библиотеку.
- Копия репозитория **perinkpheral**. Содержит `sdk/` и `tools/`.
- **Устройство** в той же сети с прошивкой, в которой включены инструменты разработки (по умолчанию). Адрес устройства показан на второй странице About. Укажите этот адрес в `PERINKPHERAL_HOST`.

Создание, сборка, запуск

```
tools/devtool/devtool.py new my-app --title "My app"     # создать папку из примера SDK
tools/devtool/devtool.py run my-app                      # собрать, загрузить, перезапустить в приложение
tools/devtool/devtool.py serve my-app                    # локальная страница: экран, кнопки, журнал, Run
```

`run` собирает приложение, загружает его и перезапускает устройство в него. Это занимает около десяти секунд. Прошивка не перезаписывается.

Сборка считает предупреждения компилятора ошибками (`-Wall -Wextra -Werror`). Неиспользуемый параметр помечается `(void)name;`.

`serve` запускает локальную веб-страницу с экраном устройства, четырьмя кнопками, журналом устройства и кнопкой Run. Клавиши со стрелками, Enter и Backspace работают как кнопки.

`new` также создаёт `CMakeLists.txt` в папке приложения. Откройте эту папку в IDE с поддержкой CMake, например CLion или VS Code.

IDE находит заголовочные файлы SDK и сгенерированные `strings.h` и `pictures.h`. В проекте три цели сборки: `my-app` (сборка), `my-app-run` (сборка и запуск на устройстве) и `my-app-serve` (страница с экраном).

Задайте `WASI_SDK_PATH` и `PERINKPHERAL_HOST` в окружении CMake в IDE.

Пример приложения

```
#include "app_text.h"
#include "host_api.h"

#define MARGIN 16

static const char PRESSES_KEY[] = "presses";
#define PRESSES_KEY_LENGTH ((int32_t)sizeof PRESSES_KEY - 1)

static int32_t presses;

static void draw_line(int32_t line, const char *text)
{
    int32_t line_height = gfx_font_height(HOST_FONT_TEXT);
    gfx_text(MARGIN, MARGIN + line * line_height, HOST_FONT_TEXT, text, app_text_length(text));
}

void app_init(void)                 /* загрузить состояние из хранилища */
{
    kv_get(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_terminate(void)         /* сохранить состояние перед выгрузкой */
{
    kv_set(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_button(int32_t event)   /* изменить состояние, запросить перерисовку */
{
    switch (event) {
    case BTN_UP_PRESSED:
    case BTN_OK_PRESSED:
    case BTN_DOWN_PRESSED:
        presses++;
        request_redraw(HOST_REDRAW_NORMAL);
        break;
    }
}

void app_on_draw(void)              /* нарисовать весь кадр по состоянию */
{
    char       counter_buffer[32];
    app_text_t counter;
    app_text_start(&counter, counter_buffer, sizeof counter_buffer);
    app_text_add(&counter, "Presses: ");
    app_text_add_number(&counter, presses);

    draw_line(0, "Hello from wasm");
    draw_line(2, counter_buffer);
}
```

BACK здесь обрабатывается хостом: короткое нажатие BACK закрывает приложение, у которого нет экспорта `app_on_back()`.

Отчёт сборки

```
tools/mkapp/mkapp.py my-app --out build/my-app

build/my-app/app.wasm: 811 bytes, 5 host calls
  module  0.8 KB of 48.0 KB
  code    0.5 KB; running it takes about 13.0 KB of the interpreter's 64.0 KB
  memory  9.1 KB of 64.0 KB: 8.0 KB of stack, 1.1 KB of data; 54.9 KB left for app_alloc()
```

Строка	Что измеряет	При превышении
`module`	Размер файла: код и начальные данные. Исполняется из флеш-памяти	Перенесите картинки и таблицы в `files/`. Используйте Kit вместо собственной отрисовки
`code`	Размер кода и оценка памяти интерпретатора, нужной для исполнения всего кода. **Самый дефицитный ресурс**: один байт кода занимает четыре байта памяти	См. «Память» в разделе 13
`memory`	Страница 64 КБ: стек, статические данные и остаток для арены	Выделяйте большие буферы через `app_alloc()` под отметкой. Читайте файлы частями

Журнал сообщает о двух состояниях работающего приложения, один раз на экземпляр или в минуту: обработчик выполняется дольше 400 мс (`slow handler: on_draw took 520 ms`), и более 12 записей в хранилище за минуту. Каждая запись в хранилище - запись во флеш: состояние сохраняется в `app_on_terminate()`.

Сборка также завершается ошибкой в следующих случаях:

- модуль импортирует функцию, не объявленную в `host_api.h`;
- код содержит операцию с плавающей точкой (отчёт её называет);
- код вызывает функцию `sys_*`, а в манифесте нет `"system": true`;
- манифест превышает лимит;
- перевод не соответствует `en.json` (раздел 12);
- функция названа как обработчик (`app_on_...`), а такого обработчика у хоста нет;
- код вызывает `timer_set()`, `alarm_set*()` или `http_request()` и не содержит обработчика, который получает результат;
- манифест объявляет опрос или интенты, а в коде нет `app_on_values()`, `app_on_data()` или `app_on_intent()`;
- код рисует и не содержит `app_on_draw()`;
- строка из `strings/` или картинка из `pictures/` не используется в коде.

5 Пакет

```
my-app/
  manifest.json      идентификация, опросы, настройки
  my-app.c           исходный код: один или несколько файлов .c
  strings/           необязательно: en.json, ru.json ...; один файл на язык    (раздел 12)
  pictures/          необязательно: PNG, PBM или текстовые строки из '#' и '.'          (раздел 6)
  files/             необязательно: файлы данных, читаются через file_read()
```

`mkapp` собирает пакет из этой папки. Он компилирует исходный код с библиотекой SDK, генерирует `strings.h` из `strings/` и `pictures.h` из `pictures/` и проверяет все лимиты. Пакет содержит `app.wasm`, `manifest.json`, файлы и по одной таблице строк на язык.

Манифест

```
{
  "name": "plex",
  "title": "Plex Now Playing",
  "version": "0.1.0",
  "abi": 2,
  "hosts": ["{{server}}"],
  "polls": [
    { "url": "{{server}}/status/sessions", "every": "{{every}}",
      "headers": { "Accept": "application/json", "X-Plex-Token": "{{token}}" },
      "pick": ["MediaContainer.Metadata[0].title", "MediaContainer.Metadata[0].grandparentTitle"] }
  ],
  "settings": [
    { "key": "server", "type": "url",    "title": "Server" },
    { "key": "token",  "type": "secret", "title": "Plex token" },
    { "key": "every",  "type": "choice", "title": "Refresh",
      "options": ["15 s", "1 min", "5 min"], "default": "1 min" }
  ],
  "intents": ["started"],
  "icon": ["..........", ".#########.", ".#.......#.", ".#..#....#.", ".#..###..#.", ".#..####.#.",
           ".#..###..#.", ".#..#....#.", ".#.......#.", ".#########.", ".........."]
}
```

Поле	Формат	Описание
`name`	1 - 13 символов из `a–z 0–9 – _`; совпадает с именем папки	Идентификатор: папка на устройстве, ключ в каталоге, пространство имён хранилища. Изменённое имя - другое приложение
`title`	1 - 23 символа	Отображаемое название в меню и на веб-странице. Переводится
`version`	до 15 байт	Задаётся разработчиком. Другая версия в каталоге предлагается как обновление
`abi`	2	Версия API хоста, под которую собрано приложение. Несовпадение отклоняется при установке
`fullscreen`	true / false (по умолчанию)	false: сверху заголовок хоста, поверхность 400 x 260. true: поверхность 400 x 300
`icon`	11 строк по 11 символов `#` и `.`	Показывается в меню, на веб-странице и в каталогах. Необязательно
`polls`	до 4	Запросы по расписанию, выполняемые хостом (ниже)
`hosts`	до 4, каждый до 63 байт	Адреса, разрешённые для `http_request()`: имя, имя с портом либо `{{key}}` настройки типа `url` или `link`. Требуется только для запросов из кода; на опросы не распространяется
`настройки`	до 8	Форма настроек на веб-странице устройства (ниже)
`intents`	`started`, `time_changed`, `network_up`	События устройства, по которым загружается приложение. Неизвестное значение приводит к ошибке сборки
`system`	true / false (по умолчанию)	Включает вызовы `sys_*`. Перед установкой веб-страница предупреждает пользователя
`companion`	URL с `https`	Страница разработчика, открываемая из формы настроек, для настройки, которую нельзя выразить формой

Неизвестные ключи хост игнорирует. `mkapp` проверяет перечисленные выше ключи.

Опросы

Поле	Формат	Описание
`url`	До 511 байт. `http` или `https`; либо начинается с `{{key}}` настройки типа `url`; либо целиком `{{key}}` настройки типа `link`	`{{key}}` любой настройки заменяется сохранённым значением в URL-кодировании: каждый символ, кроме букв, цифр и `- . _ ~`, превращается в %XX, **включая `/`**. Одна настройка - один сегмент пути: для `owner/name` нужны две настройки, `/repos/{{owner}}/{{name}}`. Настройка типа `url` в начале адреса и настройка типа `link` подставляются без изменений. Опрос с пустой настройкой в URL или заголовках завершается ошибкой `HOST_FAILED_NOT_READY`; поэтому токен в заголовке обязателен
`every`	от 10 до 86400 секунд либо `''` `{{key}}''` настройки	Интервал. При использовании настройки сохранённый текст - число и единица; единица определяется по первой букве: `s`, `m`, `h`, `d` ("30 s", "15 min", "1 h", "1 day"). Число без единицы - секунды. Выбор пользователя применяется хостом. Пока пользователь не выбрал, интервал равен `default` настройки, а без него - 15 минутам; приложение не сохраняет для него значение
`headers`	до 4; значение до 255 байт	В значениях поддерживается `{{key}}`. Используется для передачи токена
`pick`	до 16 путей, каждый до 63 байт	Пути JSON, например `current.temperature_2m`, `daily.code[0]`. Значения извлекаются хостом. Без `pick` ответ передаётся потоком

Опросы нумеруются с 0 в порядке списка. Этот номер получают обработчики.

Настройки

Настройки объявляются в манифесте. Веб-страница устройства строит форму настроек только по манифесту. Каждое значение хранится как текст в хранилище приложения под ключом настройки.

Приложение читает настройку вызовом `kv_get()` по ключу настройки.

Когда пользователь сохраняет форму, хост вызывает `app_on_settings()` и сразу повторяет все опросы.

Тип	Поле формы	Хранимое значение
`text`	Строка текста	Текст, до 255 байт
`number`	Число	Цифры, необязательные знак и десятичная точка
`choice`	Выбор из `options`, до 8 вариантов	Текст варианта, 1 - 23 символа. Переводы добавляют подписи; хранится сам вариант
`toggle`	Флажок	`"1"` или `"0"`; пусто до первого сохранения
`secret`	Поле пароля	Секрет. Для веб-страницы только запись; приложение читает его через `kv_get()`
`place`	Поиск места по названию	Название места. Страница также заполняет две настройки типа `number`, указанные в `latitude` и `longitude`
`url`	Адрес сервера пользователя	Схема, хост и необязательный порт: `http://192.168.1.10:8123`
`link`	Полный URL, заданный пользователем	`https://host/path?key`, до 255 байт. На веб-странице скрыт, как секрет. `webcal://` преобразуется в `https://`

- `default` только отображается: страница показывает его, пока настройка пуста, а хранилище остаётся пустым. Код применяет то же значение по умолчанию, когда `kv_get()` не находит значения.
- Настройки типов `url` и `link` доступны приложению только для чтения: `kv_set()` по их ключам завершается ошибкой. Когда пользователь меняет такую настройку, хост стирает секреты, не введённые заново, поэтому токен остаётся с тем сервером, для которого был введён.
- `title` - от 1 до 31 символа. `help` - до 95 символов.

6 Экран

Поверхность и цвет

- Начало координат - левый верхний угол поверхности. `gfx_width()` и `gfx_height()` возвращают её размер. Используйте эти значения для компоновки.
- Вызовы отрисовки работают внутри `app_on_draw()`. В начале каждого вызова поверхность белая. В любом другом обработчике вызовы `gfx_*` и `ui_*` возвращают `HOST_ERR_NOT_DRAWING`. Отрисовка обрезается по границам поверхности.
- `gfx_color()` задаёт цвет для последующих вызовов, от `HOST_PAPER` (0) до `HOST_INK` (16). В начале каждого `app_on_draw()` цвет равен `HOST_INK`. Промежуточные значения - оттенки серого, получаемые дизерингом. Используйте серый только для заливок: тонкие линии и мелкий текст становятся нечитаемыми.
- Стандартное поле - `HOST_UI_MARGIN` (16 пикселей). Используйте его у левого и правого краёв.

Шрифты

Константа шрифта задаёт размер. Гарнитура - настройка пользователя: Departure Mono (моноширинная) или Jersey (пропорциональная). Ширина символов у них разная, поэтому компоновка строится на измеренной ширине.

Измеряйте текст вызовами `gfx_text_width()`, `gfx_font_height()`, `gfx_font_ascent()` и `gfx_font_cap_height()`.

Константа	Высота прописных, пикс.	Высота строки, пикс.	Символов на 400 пикс.	Типичное применение
`HOST_FONT_SMALL`	8 / 10	15 / 21	57 / 50	Единицы, подписи
`HOST_FONT_TEXT`	16 / 15	30 / 28	28 / 33	Основной текст; текст самого хоста
`HOST_FONT_MEDIUM`	24 / 25	45 / 43	19 / 21	Заголовки, второстепенные значения
`HOST_FONT_LARGE`	32 / 30	60 / 56	14 / 16	Строки сообщения
`HOST_FONT_DISPLAY`	48 / 50	90 / 86	9 / 10	Главное значение экрана
`HOST_FONT_HUGE`	80 / 80	150 / 140	5 / 6	Часы: пять цифр занимают всю ширину

В каждой ячейке значение для Departure Mono, затем для Jersey.

- Аргумент `y` вызова `gfx_text()` - верх строки. Базовая линия находится на `y + ascent`; верх прописных - на `y + ascent − cap_height`.
- Добавляйте `HOST_FONT_TABULAR` к шрифту для чисел, меняющихся на месте, например часов или обратного отсчёта. Все цифры получают одинаковую ширину. Флаг принимает любой вызов с аргументом шрифта. При измерении передавайте тот же флаг.
- `gfx_text_fit()` сокращает текст до заданной ширины и добавляет "...". Обрезка идёт по границе символа, поэтому результат остаётся корректным UTF-8.

Текст за пределами ASCII

Весь текст - в UTF-8. Хост рисует каждый символ первым доступным способом:

1. Глиф символа. Печатные символы ASCII доступны всегда. Латиница с диакритикой, кириллица и греческий доступны, если пользователь установил пакет глифов (Settings, Language на веб-странице устройства).
2. Транслитерация в ASCII: "Moskva", "Strasse", "Athina". Типографские тире и кавычки заменяются простыми.
3. Вопросительный знак.

Знак градуса встроен во все размеры.

Ширина текста зависит от использованного способа. Всегда измеряйте текст перед размещением.

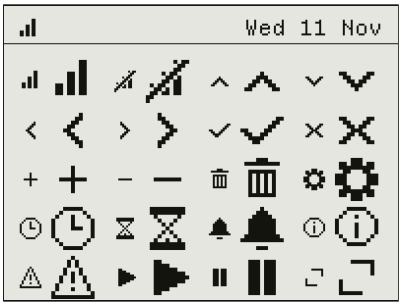
Лимиты на текст, видимый пользователю, например заголовок или вариант, заданы в символах. Поле вмещает вдвое меньше двухбайтовых символов, чем однобайтовых.

Картинки и иконки

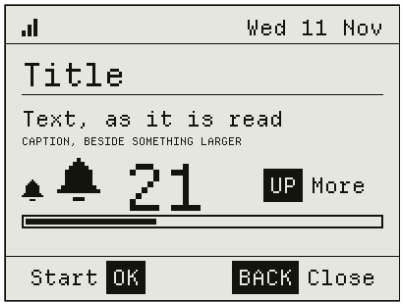
Хост предоставляет 20 встроенных иконок. Каждая иконка нарисована на сетке 11 x 11 клеток. Приложение может использовать их на своих экранах.

`gfx_icon()` рисует иконку с размером клетки от 1 до 16 пикселей. `ui_icon()` рисует иконку в одном из трёх стандартных размеров: 22, 44 или 99 пикселей.

Константа	Значение	Константа	Значение
`HOST_ICON_SIGNAL`	подключено	`HOST_ICON_NO_SIGNAL`	не подключено
`HOST_ICON_UP`	стрелка вверх	`HOST_ICON_DOWN`	стрелка вниз
`HOST_ICON_LEFT`	стрелка влево	`HOST_ICON_RIGHT`	стрелка вправо
`HOST_ICON_CHECK`	выбрано, готово	`HOST_ICON_CROSS`	отмена, закрыть
`HOST_ICON_PLUS`	добавить	`HOST_ICON_MINUS`	убрать
`HOST_ICON_TRASH`	удалить	`HOST_ICON_GEAR`	настройки
`HOST_ICON_CLOCK`	время	`HOST_ICON_HOURGLASS`	ожидание, таймер
`HOST_ICON_BELL`	уведомление, будильник	`HOST_ICON_INFO`	информация
`HOST_ICON_WARNING`	предупреждение, ошибка	`HOST_ICON_PLAY`	запуск, воспроизведение
`HOST_ICON_PAUSE`	пауза	`HOST_ICON_UPDATE`	обновление; знак проекта



Встроенные иконки при размере клетки 2 и 4 пикселя



Базовые элементы Kit: заголовок, текст, подпись, иконки, значение, кнопка, полоса, строка кнопок

Собственные картинки приложения размещаются в папке `pictures/`. Поддерживаются форматы PNG, PBM и текстовый. Текстовый файл содержит строки из `#` (чернила) и `.` (бумага) и может содержать несколько именованных картинок:

```
// pictures/things.txt
drop:
.....#.....
....###....
...#####...
..#######..
```

```
#include "pictures.h"   /* генерируется mkapp */

gfx_picture(x, y, PICTURE_DROP, 4);    /* 44 пикс.: размер клетки 4 */
gfx_picture(x, y, PICTURE_DROP, 2);    /* 22 пикс.: размер клетки 2 */
```

Правила для текстовых файлов:

- Строка вида `name:` начинает картинку.
- Строки до первой такой строки образуют картинку с именем файла.
- Все строки картинки имеют одинаковую длину.
- Картинка не превышает 255 клеток в ширину и 255 клеток в высоту.

Константа - `PICTURE_` и имя картинки в верхнем регистре. Символы, кроме букв и цифр, заменяются на `_`. Пример: `half-moon` превращается в `PICTURE_HALF_MOON`.

`mkpicture.py --show` выводит картинки в том виде, как их прочитал инструмент.

Картинка хранится по одному биту на клетку. `gfx_picture()` рисует её с размером клетки от 1 до 16 пикселей, поэтому одну картинку можно использовать в нескольких размерах.

Используйте те же сетки, что у встроенных иконок: 11 клеток либо 22 и 44 клетки для большей детализации. Тогда картинки совпадают с иконками по насыщенности.

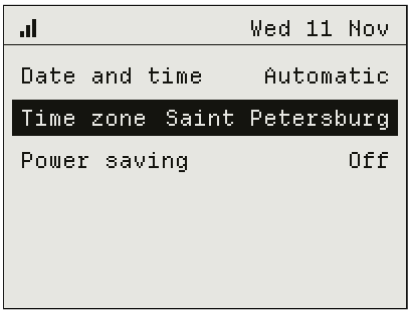
Картинку, слишком большую для модуля, размещайте в `files/` и рисуйте вызовом `gfx_bitmap_file()`. Хост читает файл напрямую; память приложения картинка не занимает.

Kit

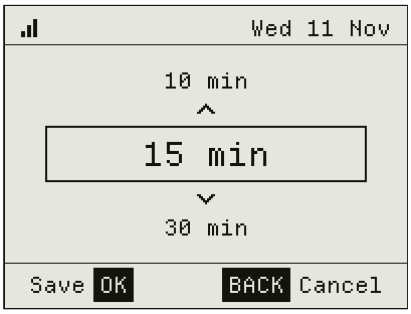
Kit - набор элементов интерфейса, из которых собраны экраны самого хоста. Приложения могут использовать те же элементы через вызовы `ui_*`.

Использовать Kit необязательно: вызовами `gfx_*` можно нарисовать что угодно. У Kit три преимущества. Приложение выглядит как остальная система. Оно следует за будущими изменениями внешнего вида без пересборки. Оно меньше, потому что код отрисовки находится в хосте.

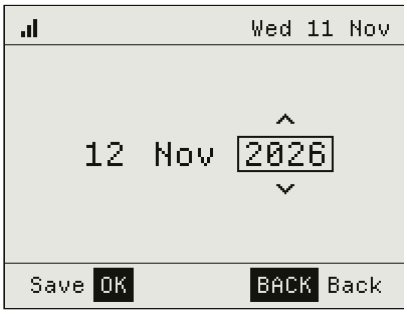
Вызов	Описание	Применение
`ui_list_row, ui_list_first, ui_list_visible`	Список: одна строка на элемент, значение или иконка у правого края, выбранная строка инвертирована, полоса прокрутки. Без выбора UP и DOWN прокручивают список	Более трёх элементов; меню; настройки
`ui_cell, ui_cell_fit`	Название, картинка и значение в рамке любого размера; значение - наибольшим шрифтом, который помещается	Панели с одним, двумя или четырьмя значениями
`ui_value`	Значение с единицей на одной базовой линии: "18 km/h"	Любое число с единицей
`ui_notice`	Две крупные строки, строка текста, крупная иконка, подписи кнопок	Вопросы, ошибки, пустые состояния, выполнение работы
`ui_reason`	Стандартное сообщение для статуса запроса: "Loading", "Not set up", "No network", "Access denied", "No answer"	Любой экран, зависящий от сети
`ui_actions, ui_button`	Строка кнопок внизу: действие OK слева, действие BACK справа	Экраны, где действие кнопки нужно подписать
`ui_picker`	Одно значение из ряда: выбранное в рамке, соседние сверху и снизу	Выбор из нескольких значений кнопками UP и DOWN
`ui_date`	Время, дата или оба; за раз задаётся одна часть	Ввод момента времени
`ui_frame`	Рамка со стрелками доступных направлений	Отмечает значение, изменяемое кнопками
`ui_bar`	Заполненная слева доля от целого	Количества. Меняется вместе с данными
`ui_chart`	Ряд целых чисел в виде линии, области, столбцов или ступеней, масштабированный под прямоугольник	История одного значения. Изменяется вместе с данными
`ui_title, ui_text, ui_text_lines, ui_icon`	Заголовок страницы с линейкой; текст в четырёх стилях, в одну строку или с переносом; иконки трёх размеров	Базовые элементы страницы



ui_list_row: список строк со значениями



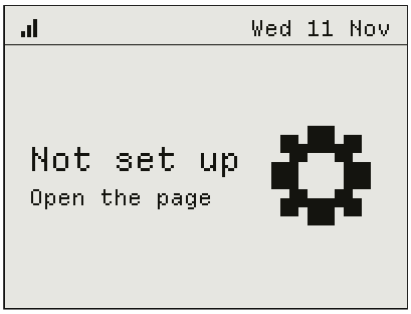
ui_picker: выбор одного значения



ui_date: установка даты или времени



ui_notice: вопрос



ui_notice: сообщение с причиной



ui_notice с is_working: выполняется работа

Обработка кнопок для списков, выбора значения и дат реализована в библиотеке SDK (`app_ui.h`). Приложение вызывает её из `app_on_button()` и действует по результату. См. раздел 7.

Приложение `kit` из каталога показывает каждый элемент на отдельной странице. Его исходный код - рабочий пример для каждого элемента.

Запрос обновления экрана

Флаг	Поведение хоста	Применение
`HOST_REDRAW_NORMAL`	Из обработчика кнопки: немедленно. Из обработчика таймера, будильника или данных: когда позволяет политика обновления, по умолчанию раз в минуту	По умолчанию
`HOST_REDRAW_SOON`	Немедленно, без нажатия кнопки	Данные, которых ждёт пользователь, например результат ручного обновления
`HOST_REDRAW_CLEAN`	Полное обновление: 2 - 3 с, одно мигание, убирает остаточное изображение	Изображения, которые должны быть чистыми; конец последовательности кадров
`HOST_REDRAW_NO_BLINK`	Немедленно; исключается из периодического полного обновления	Навигация по списку; значения, меняющиеся каждую секунду

Число запросов перерисовки не ограничено.

Каждое обновление расходует заряд батареи и изнашивает панель. Частые изменения также делают устройство отвлекающим. Рекомендации - в разделе 13.

7 Кнопки

Кнопки по порядку: UP, OK, DOWN, BACK. Хост использует их одинаково на всех экранах. Приложениям следует сохранять эти значения.

Каждое нажатие порождает события двух видов. Низкоуровневые события - `KEYDOWN` и `KEYUP` . Интерпретированные события - `PRESSED` , отправляется по окончании короткого нажатия, и `LONG_PRESSED` , отправляется через 600 мс, пока кнопка ещё удерживается. Длинное нажатие не порождает `PRESSED` .

Большинство приложений обрабатывает только `PRESSED` и `LONG_PRESSED` .

Значение события объединяет кнопку и действие. У каждой комбинации есть константа:

```
void app_on_button(int32_t event)
{
    switch (event) {
    case BTN_OK_PRESSED:       start();  break;
    case BTN_OK_LONG_PRESSED:  reset();  break;
    }
    request_redraw(HOST_REDRAW_NORMAL);
}
```

BACK

Приложение	Короткое нажатие BACK
Без `app_on_back()`	Закрывает приложение (один шаг вниз по стеку)
С `app_on_back()`	Доставляется в `app_on_back()`. Типичная обработка: вернуться на страницу назад; на первой странице вызвать `request_exit()`

Короткое нажатие BACK передаётся в `app_on_back()` либо закрывает приложение, если такого экспорта нет. Длинное нажатие BACK открывает меню хоста. `app_on_button()` получает все остальные события.

`request_exit()` снимает приложение с вершины стека. Пользователь возвращается на предыдущий экран. Это относится и к приложению, занявшему экран через `request_focus()`.

Списки, выбор значения, даты

```
#include "app_ui.h"

static app_list_t list = { .count = 12 };          /* .selected = -1: без выбора, строки прокручиваются */

void app_on_button(int32_t event)
{
    switch (app_list_button(&list, event)) {
    case APP_UI_DONE:      open_item(list.selected);               break;   /* OK */
    case APP_UI_CHANGED:   request_redraw(HOST_REDRAW_NO_BLINK);   break;   /* UP, DOWN */
    default:               break;
    }
}

void app_on_draw(void)
{
    int32_t first = app_list_first(&list);
    for (int32_t row = first; row < first + ui_list_visible(&list) && row < list.count; row++) {
        ui_list_row(&list, row, label_of(row), label_length(row), "", 0, HOST_ICON_NONE);
    }
}
```

`app_picker_t` и `app_date_t` устроены так же. Приложение хранит структуру, передаёт события кнопок в одну функцию и рисует другой. Функция обработки кнопок возвращает *changed*, *done*, *cancelled* или *ignored*.

8 Хранение

Хранилище

`kv_get()` и `kv_set()` читают и записывают небольшие значения по ключу. У каждого приложения собственное пространство имён.

В хранилище находятся состояние приложения и его настройки. Данные сохраняются при удалении приложения и снова доступны после переустановки.

```
typedef struct { int32_t version; int32_t minutes; int64_t ends_at; } state_t;
static state_t state;

void app_init(void)
{
    if (kv_get("state", 5, &state, sizeof state) != sizeof state || state.version != 2) {
        state = (state_t){ .version = 2, .minutes = 25 };     /* нет сохранённого состояния или старая версия */
    }
}
```

- Добавляйте поле версии в каждую сохраняемую структуру и проверяйте его при загрузке. Новая версия приложения будет читать данные, записанные старой.
- Каждая запись изнашивает флеш-память. Записывайте, только когда данные изменились. Не записывайте по каждому срабатыванию таймера.
- `kv_set()` с длиной 0 удаляет ключ.

Файлы

Используйте файлы для данных, не помещающихся в значение хранилища: загруженный список, кэш, картинка.

`file_read()` и `file_write()` принимают смещение. Файл можно читать и дописывать частями, поэтому держать его в памяти целиком не требуется.

Файлы из папки `files/` пакета устанавливаются вместе с приложением.

9 Сеть

Весь доступ в сеть идёт через хост. Приложение описывает запрос; хост выполняет его и доставляет ответ. Хост выполняет один запрос за раз на всё устройство. Доставка не зависит от того, находится ли приложение на экране.

О доступности сети сообщает ответ: запрос, сделанный без сети, завершается ошибкой `HOST_FAILED_NO_NETWORK`.

Способ	Запуск	Доставка
Опрос с `pick`	Расписание	`app_on_values()`, один раз, с извлечёнными значениями
Опрос без `pick`	Расписание	`app_on_data()` фрагментами, с номером опроса
`http_request()`	Код: после нажатия кнопки или как последующий запрос	`app_on_data()` фрагментами, с тегом, выбранным приложением (16 - 255)

Значения, извлечённые хостом

```
#include "app_values.h"
enum { VALUE_TEMPERATURE, VALUE_CODE };            /* порядок "pick" в манифесте */

void app_on_values(int32_t request, int32_t http_status)
{
    int32_t tenths, code;
    last_status = http_status;                     /* для ui_reason(), когда данных нет */
    if (http_status == 200 &&
        app_value_number(VALUE_TEMPERATURE, 1, &tenths) &&      /* "21.46" -> 215 */
        app_value_number(VALUE_CODE, 0, &code)) {
        keep(tenths, code);                        /* сохранить в хранилище */
    }                                              /* иначе оставить прежние данные */
    request_redraw(HOST_REDRAW_NORMAL);
}
```

Числа - целые. `app_value_number()` возвращает значение, умноженное на запрошенное число десятичных разрядов, с округлением.

У пути, отсутствующего в ответе, нет значения: `app_value_number()` возвращает false. Отсутствующее значение означает отсутствие данных.

Потоковые ответы

```
void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)
{
    if (flags & HOST_DATA_FAILED) {                /* один вызов, без данных: http_status — код HOST_FAILED_* */
        last_status = http_status;
        return;
    }
    if (flags & HOST_DATA_FIRST) {                 /* с первым фрагментом приходит статус HTTP */
        last_status = http_status;
        begin_reading();                           /* сбросить разборщик */
    }
    read_piece(data, length);                      /* данные действительны до возврата из обработчика */
    if (flags & HOST_DATA_LAST) {
        take_what_was_read();                      /* ответ получен целиком */
        request_redraw(HOST_REDRAW_NORMAL);
    }
}
```

- Обрабатывайте каждый фрагмент при получении и сохраняйте только данные для отображения. Используйте конечный автомат, `json_pick` из SDK или собственный разборщик. Ответ может занимать несколько мегабайт; у приложения 64 КБ.
- Заменяйте отображаемые данные только по `HOST_DATA_LAST`. Доставка может прекратиться без последнего фрагмента и без ошибки (раздел 3). `HOST_DATA_FIRST` всегда начинает новый ответ.
- Внутри обработчика `data_response()` возвращает метаданные ответа: статус, Content-Length, Content-Type, ETag, Last-Modified, Location, Link и Retry-After.

Запросы из кода

```
enum { REQUEST_TOGGLE = HOST_REQUEST_FIRST };
static const char URL[]     = "{{server}}/api/services/light/toggle";
static const char HEADERS[] = "Authorization: Bearer {{token}}\nContent-Type: application/json";

http_request(REQUEST_TOGGLE, HOST_HTTP_POST, URL, sizeof URL - 1, HEADERS, sizeof HEADERS - 1,
             body, body_length);
```

- Запросы разрешены только на адреса, перечисленные в манифесте. Подстановки `{{key}}` работают в URL и в значениях заголовков, поэтому коду не нужно читать токен.
- `HOST_ERR_NO_SPACE` означает, что очередь запросов устройства заполнена (два ожидающих запроса). Повторите позже.
- Для `GET` хост выполняет до трёх перенаправлений. Перенаправление на адрес вне списка завершается ошибкой `HOST_FAILED_NOT_ALLOWED`. Запрос с телом отправляется один раз; ответ с перенаправлением доставляется как ответ на запрос.
- `poll_now(n)` немедленно выполняет опрос `n`, например когда пользователь нажимает кнопку обновления. `poll_set_interval(n, seconds)` меняет интервал опроса из кода, например чтобы учесть заголовок Retry-After.

Неудавшиеся запросы

Статус	Причина
`HOST_FAILED_NO_NETWORK (-1)`	Нет подключения к сети
`HOST_FAILED_NO_ANSWER (-2)`	Нет ответа от сервера либо ответ неполный
`HOST_FAILED_NOT_READY (-3)`	`{{setting}}`, используемая запросом, пуста
`HOST_FAILED_NOT_ALLOWED (-4)`	Перенаправление на адрес вне списка манифеста
`HOST_FAILED_TIMEOUT (-5)`	Истекло время ожидания после соединения с сервером
`HOST_FAILED_NOT_TRUSTED (-6)`	Устройство не доверяет сертификату HTTPS
`HOST_FAILED_TOO_LARGE (-7)`	Ответ больше 256 КБ (извлечение значений) или 4 МБ (потоком)
`HOST_FAILED_NO_MEMORY (-8)`	Недостаточно памяти устройства для запроса

Сохраняйте последний статус в хранилище. Когда данных для отображения нет, передавайте статус в `ui_reason()`. Вызов рисует стандартное сообщение для статуса и возвращает, было ли что-то нарисовано:

Статус	Сообщение	Возвращаемое значение
`0 (запроса ещё не было)`	Loading	1
`HOST_FAILED_*`	По одному сообщению на код: Not set up, No network, No answer ...	1
`401, 403`	Access denied / Check the settings	1
`404`	Not found / Check the address	1
`Любой другой статус, кроме 2xx`	No answer / Code <status>	1
`200 – 299`	Нет. Ответ получен; об отсутствии данных сообщает приложение	0

10 Фоновая работа

Хост загружает приложение, которое не на экране, для одного вызова при наступлении одного из событий:

Задача	Механизм	Примечания
Данные по расписанию	Опрос в манифесте	Действует после перезагрузки без кода
Действие в заданное время	`alarm_set, alarm_set_at, alarm_set_every`	Будильники сбрасываются при перезагрузке. Восстанавливайте их в интенте `started` по времени из хранилища
Сообщение пользователю	`notify`	Показывается хостом поверх текущего экрана, пока не будет закрыто
Вывод приложения на экран	`request_focus`	Разрешается хостом, не чаще раза в 30 с
Статус в заголовке	`header_status`	Иконка и короткий текст в заголовке над любым экраном, пока приложение не на экране
Реакция на запуск устройства, подключение к сети или изменение часов	Интент в манифесте	`app_on_intent()`

Уведомления

```
void app_on_alarm(int32_t id)                    /* приложение загружено для этого вызова */
{
    host_notice_t notice = { .first = "Time", .second = "is up", .icon = HOST_ICON_BELL };
    notify(&notice);
}
```

После возврата из `notify()` уведомлением управляет хост. Он показывает сообщение под заголовком, держит его, пока пользователь не нажмёт OK или BACK, а затем восстанавливает предыдущий экран.

Уведомлением полностью управляет хост. Пока оно показано, приложение остаётся выгруженным.

Используйте `notify()` вместо `request_focus()`, когда сообщение помещается в две строки.

Будильники, таймеры

Вызов	Поведение
`timer_set(id, milliseconds)`	Однократный. Вызывает `app_on_timer(id)`. Действует, только пока приложение на экране; устанавливайте заново в `app_on_focus()`
`alarm_set(id, seconds)`	Однократный. Вызывает `app_on_alarm(id)` на экране и вне его. Отсчитывает прошедшее время, не зависит от изменений часов
`alarm_set_at(id, unix_time)`	Однократный, по времени часов. Следует за изменениями часов
`alarm_set_every(id, seconds, first_in)`	Повторяющийся, с периодом от 60 с до 1 суток. Каждый вызов загружает приложение; используйте наибольший подходящий период

11 Время

- `time_now()` возвращает время Unix в секундах. Возвращает 0, пока устройство не получило время. Приложение обязано обрабатывать этот случай.
- Отображайте местное время. Часовым поясом управляет хост. `time_local()` возвращает текущие местные дату и время, `time_local_at()` преобразует любой момент, `time_from_local()` преобразует местные дату и время обратно во время Unix, например для `alarm_set_at()`.
- `time_text()` форматирует момент времени как текст на языке устройства. Собственные названия дней и месяцев приложениям не нужны. Форматы: "15:04", "Fri 13 Nov", "Friday", "13 November" и относительная форма: "15:04", "Tmrw 15:04", "Sat 15:04", "21 Nov".
- Чтобы получать вызов при установке или изменении часов, объявите интент `time_changed`.

12 Языки

Приложение отображает свой текст на языке устройства. Переводы предоставляет разработчик. Хост загружает перевод для текущего языка.

```
strings/en.json    { "no_events": "No events", "starts_in": "In {0} min" }
strings/ru.json    { "no_events": "Нет событий", "starts_in": "Через {0} мин",
                     "manifest": { "title": "Календарь",
                                   "settings": { "every": { "title": "Обновление",
                                                            "options": { "15 min": "15 мин" } } } } }
```

```
#include "strings.h"                              /* генерируется mkapp: STR_NO_EVENTS, STR_STARTS_IN, app_string() */

const char *text = app_string(STR_NO_EVENTS);     /* текст на языке устройства */

const host_text_argument_t minutes = { .number = 10 };
char line[32];
int32_t length = text_fill(app_string(STR_STARTS_IN), app_text_length(app_string(STR_STARTS_IN)),
                           &minutes, 1, line, sizeof line);          /* "In 10 min" */
```

- `en.json` обязателен. Английский встраивается в модуль и используется, когда перевода нет. Все остальные файлы содержат точно те же ключи. Ключ `manifest` допускается только в переводах (см. ниже).
- При загрузке приложения хост копирует таблицу строк на языке устройства в память приложения поверх английской. После этого `app_string()` возвращает переведённый текст без дополнительного вызова.
- `text_fill()` заменяет `{0}`, `{1}` в строке числами или текстами. Перевод может располагать их в любом порядке. Пишите каждую строку в одной форме для всех чисел, например "min".
- Ключ `manifest` перевода содержит переводимые части манифеста: название, заголовок и пояснение каждой настройки, подписи вариантов выбора. Хранимое значение варианта не меняется; переводится только его подпись.
- Сборка завершается ошибкой, если языка нет в `sdk/languages.json` (27 языков, двухбуквенные коды), если в переводе не хватает ключа или есть лишний, либо если строка содержит символ, которого нет в пакетах глифов языка.
- Переводы обычно примерно на треть длиннее английского текста. Kit сокращает текст, который не помещается, с "...". Проверяйте переведённые экраны на устройстве.

13 Правила проектирования

Эти правила относятся ко всем приложениям. Экраны самого хоста им следуют. Большинство из них - соглашения; хост проверяет лишь некоторые.

Содержимое экрана

Показывайте одно главное значение на экране.
Устройство читают примерно с метра, примерно за секунду. Выберите главное значение, нарисуйте его самым крупным, остальное оставьте мелким.

Делайте экраны статичными.
Меняйте экран, только когда меняются его данные. Полоса показывает количество. Для индикации работы показывайте сообщение с `is_working`.

Перерисовывайте только при изменении содержимого.
Таймер должен запрашивать перерисовку, только если изображение изменится. Часы перерисовываются раз в минуту, на границе минуты. По умолчанию хост допускает одно обновление в минуту без нажатия кнопки. Проектируйте под эту частоту.

Рисуйте чёрным по белому.
Используйте чернила на бумаге. Серый используйте только для заливок. Белый текст на чёрной полосе используйте только для выбранного элемента или новой информации.

Делайте текст интерфейса коротким и стандартным.
Используйте общепринятые слова: "Save", "Remove", "Not set up". Если подпись требует пояснения, смените подпись.

Память

Сокращайте сначала размер кода, затем размер данных.
Каждый байт исполненного кода занимает около четырёх байт из 64 КБ интерпретатора. Каждый байт данных занимает один байт страницы 64 КБ, которая в основном свободна. Предпочитайте таблицу цепочке операторов `if`, Kit - собственной отрисовке, `pick` - собственному разборщику, `time_text()` - собственному форматированию дат.

Не встраивайте общие вспомогательные функции.
Небольшую вспомогательную функцию, вызываемую из многих мест, помечайте `__attribute__((noinline))` и сравнивайте отчёты сборки.

Храните картинки в битах.
Размещайте картинки в `pictures/`. Большие картинки размещайте в `files/`.

Обрабатывайте ответы потоком.
Обрабатывайте ответ по мере поступления и сохраняйте только данные для отображения. Выделяйте рабочие буферы через `app_alloc()` и освобождайте их по отметке.

Состояние и время

Сохраняйте состояние перед каждой выгрузкой.
Всё значимое состояние должно быть в хранилище к моменту возврата из `app_on_terminate()`. Проверка: откройте приложение, выйдите, откройте снова. Экран должен выглядеть так же.

Сохраняйте показанные данные, пока новые не получены целиком.
После неудавшегося или прерванного запроса оставляйте на экране прежние данные.

Объясняйте пустой экран.
Вызывайте `ui_reason()`, чтобы показать "Not set up", "No network" или "Loading".

Обрабатывайте неизвестное или изменённое время.
Обрабатывайте `time_now() == 0`. Используйте `alarm_set()` для задержки («через десять минут») и `alarm_set_at()` для времени по часам («в семь»).

Взаимодействие и данные

Принимайте текстовый ввод на веб-странице.
Текст - токен, адрес, город - объявляйте настройкой в манифесте; пользователь вводит его на веб-странице устройства. На устройстве давайте пользователю выбор из нескольких значений кнопками UP и DOWN.

Занимайте экран редко.
Используйте уведомление только для событий, о которых пользователь просил сообщать, например таймер или встреча. Остальная информация ждёт, пока пользователь посмотрит на устройство.

Отправляйте данные пользователя только туда, куда он указал.
Перечисляйте только те адреса, которые нужны приложению. Для собственного сервера пользователя используйте настройку типа `url`. Для токена используйте настройку типа `secret` и передавайте его подстановкой в заголовке.

Опрашивайте не чаще, чем меняются данные.
Предлагайте интервал как настройку с разумным значением по умолчанию. При питании от батареи хост ограничивает каждый опрос одним разом в пять минут.

14 Инструменты и отладка

Команда	Назначение
`tools/mkapp/mkapp.py <dir> --out <dir>`	Собирает одно приложение: компиляция, компоновка с библиотекой SDK, проверка, измерение
`tools/devtool/devtool.py new <dir> [--title T]`	Создаёт папку нового приложения из примера SDK
`tools/devtool/devtool.py run <dir> [--shot f.png]`	Собирает, загружает и перезапускает устройство в приложение
`tools/devtool/devtool.py serve [dir]`	Локальная веб-страница: экран устройства, кнопки, журнал, Run
`tools/devtool/devtool.py shot [f.png]`	Сохраняет текущий кадр панели как изображение

Команда	Назначение
`tools/devtool/devtool.py press 3 3 2L [--shot f.png]`	Отправляет нажатия кнопок: 1 - 4 короткие, 1L - 4L длинные, в порядке кнопок устройства
`tools/devtool/devtool.py open <name>`	Открывает установленное приложение
`tools/devtool/devtool.py log [--follow] [--app]`	Выводит журнал устройства. `--app`: только строки приложений
`tools/devtool/devtool.py set <app> <key> <value>`	Записывает значение в хранилище приложения, например настройку
`tools/devtool/devtool.py status`	Выводит прошивку, сеть, свободную память и причину последней перезагрузки
`tools/mkpicture/mkpicture.py <files> --show`	Выводит картинки строками из # и . в том виде, как их читает `mkapp`
`tools/mkvideo/mkvideo.py <video> --out <bin>`	Преобразует видео в однобитный поток, помещающийся в файл приложения
`tools/catalog/mkcatalog.py <folder> --out <dir>`	Собирает папку приложений в каталог

`host_log(level, text, length)` записывает одну строку до 200 символов в журнал устройства. Заменяет `printf`.

`press ... --shot` дожидается окончания обновления панели и сохраняет кадр. Это позволяет автоматизировать регрессионные тесты экранов приложения.

Типичные проблемы

Признак	Вероятная причина
Пустой экран	Отрисовка вне `app_on_draw()` либо отсутствие `request_redraw()` после изменения состояния
Состояние теряется после ухода и возврата	Состояние хранится в глобальных переменных. Сохраняйте его в `app_on_terminate()`, загружайте в `app_init()`
"<title> crashed" с "memory alloca..."	Исполненный код не помещается в память интерпретатора. См. предупреждение сборки и «Память» в разделе 13
"<title> cannot start", "Update the app."	Собрано с другим `host_api.h`, чем прошивка устройства. Пересоберите и переустановите
Опрос завершается с -3	`{{setting}}` в его URL или заголовках пуста
HTTPS завершается с -6	Самоподписанный или просроченный сертификат сервера. В домашней сети используйте `http`
`http_request()` возвращает -4	Очередь запросов устройства заполнена. Повторите из следующего обработчика
Остаточное изображение предыдущих кадров	Последовательность перерисовок `NO_BLINK` без завершающей перерисовки `CLEAN`
Символы показаны как ? или транслитерированы	Пакет глифов для этой письменности не установлен либо текст обрезан по байтам
Нажатие игнорируется вскоре после запуска	Шла фоновая доставка. Нажатие прервало её, экран перерисовывается

15 Публикация

Устройство устанавливает приложения из *каталогов*. Каталог - папка статических файлов с индексом. Его можно разместить на любом хостинге с HTTPS, например в виде файлов git-репозитория или статического сайта.

```
tools/catalog/mkcatalog.py my-apps --out catalog       # собрать все приложения в папке
                                                       # результат: catalog/catalog.json и по папке на приложение
```

- В индексе перечислены имя, название, версия и иконка каждого приложения, а также его файлы с размерами и хешами SHA-256. Устройство принимает файл, только если совпадают и размер, и хеш.
- Модель доверия: HTTPS до источника, выбранного пользователем, плюс песочница. Пакеты не подписываются.

- Официальный каталог - репозиторий `perinkpheral-apps`. Это первый источник на каждом устройстве. Пользователь может добавить ещё до трёх источников на веб-странице устройства, указав адрес репозитория.
- Веб-страница показывает сводку по каждому установленному приложению, взятую из манифеста: серверы, к которым оно подключается, интервал опроса, что оно хранит и запускается ли вместе с устройством.
- Увеличивайте `version` при каждом изменении. Устройство сравнивает версии, чтобы обнаружить обновления.

Часть III. Справочник

В этой части перечислены все объявления `sdk/host_api.h` для ABI 2.

Числовые аргументы и возвращаемые значения имеют тип `int32_t`, если не указано иное. Текстовый аргумент - указатель и длина в байтах; текст в UTF-8.

Вызов без собственного возвращаемого значения возвращает `HOST_OK` или отрицательный код `HOST_ERR_*`.

16 Коды возврата

Код	Значение
`HOST_OK (0)`	Успех
`HOST_ERR_NOT_DRAWING (-1)`	Вызов отрисовки вне `app_on_draw()`
`HOST_ERR_ARGUMENT (-2)`	Указатель вне памяти приложения; неизвестный шрифт, иконка, таймер или стиль; размер вне диапазона
`HOST_ERR_NOT_FOUND (-3)`	Ключ не сохранён; файл не найден; время не установлено; путь отсутствует в ответе
`HOST_ERR_NO_SPACE (-4)`	Значение больше буфера; хранилище или файлы заполнены; очередь запросов или уведомлений заполнена
`HOST_ERR_FAILED (-5)`	Сбой хранилища

17 Экспорты

Все экспорты необязательны. Обработчик, который модуль не экспортирует, хост пропускает.

Экспорт	Когда вызывается
`void app_init(void)`	Экземпляр загружен, его строки на месте. Один раз на экземпляр
`void app_on_focus(void)`	Приложение получает экран. Здесь запускаются таймеры `timer_set()`
`void app_on_blur(void)`	Приложение сейчас потеряет экран
`void app_on_terminate(void)`	Экземпляр сейчас будет выгружен по любой причине: уход с экрана, конец фонового вызова, перезагрузка устройства, удаление. Если приложение было на экране, перед этим вызывается `app_on_blur()`
`void app_on_draw(void)`	Нужен кадр. Нарисуйте весь кадр по состоянию. Поверхность белая
`void app_on_button(int32_t event)`	Происходит событие кнопки: `BTN_<UP|OK|DOWN|BACK>_<KEYDOWN|KEYUP|PRESSED|LONG_PRESSED>`. Кроме `BTN_BACK_PRESSED` и `BTN_BACK_LONG_PRESSED`
`void app_on_back(void)`	Короткое нажатие BACK. Для приложений с внутренними страницами
`void app_on_timer(int32_t id)`	Срабатывает таймер
`void app_on_alarm(int32_t id)`	Срабатывает будильник, на экране или вне его
`void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)`	Приходит фрагмент ответа. `request` - номер опроса или тег запроса. `flags`: `HOST_DATA_FIRST`, `_LAST`, `_FAILED`

Экспорт	Когда вызывается
`void app_on_values(int32_t request, int32_t http_status)`	Приходит ответ на опрос с `pick`. Значения доступны внутри обработчика
`void app_on_settings(void)`	Пользователь сохраняет настройки приложения на веб-странице
`void app_on_intent(int32_t intent)`	Происходит объявленное событие устройства: `HOST_INTENT_TIME_CHANGED` (1), `_NETWORK_UP` (2), `_STARTED` (3)
`uint8_t *app_data_buffer(void)`	Возвращает буфер для одного фрагмента данных (1 КБ). Предоставляется `sdk/lib`
`uint8_t *app_strings_buffer(void)`	Возвращает буфер для строк на языке устройства. Генерируется `mkapp`

18 Отрисовка

Вызов	Описание
`gfx_width(void), gfx_height(void)`	Размер поверхности: 400 x 260 под заголовком, 400 x 300 на весь экран
`gfx_color(color)`	Задаёт цвет отрисовки: от `HOST_PAPER` (0) до `HOST_INK` (16)
`gfx_text(x, y, font, text, length)`	Рисует текст; y - верх строки. Возвращает ширину. До 127 байт
`gfx_text_width(font, text, length)`	Возвращает ширину текста
`gfx_text_fit(font, text, length, max_width, fitted, capacity)`	Сокращает текст с "..." до заданной ширины. Возвращает новую длину
`gfx_font_height(font), gfx_font_ascent(font), gfx_font_cap_height(font)`	Высота строки; расстояние от верха строки до базовой линии; высота прописных и цифр
`gfx_rect(x, y, w, h, fill)`	Прямоугольник: `HOST_RECT_OUTLINE` или `HOST_RECT_FILLED`
`gfx_line(x0, y0, x1, y1)`	Линия шириной 1 пиксель, включая концы
`gfx_picture(x, y, picture, cell)`	Картинка, сгенерированная `mkapp` из `pictures/`: байт ширины, байт высоты, один бит на клетку. Размер клетки 1 - 16 пикселей
`gfx_icon(icon, x, y, cell)`	Встроенная иконка `HOST_ICON_*`, 11 x 11 клеток. Размер клетки 1 - 16 пикселей
`gfx_bitmap(x, y, w, h, bits)`	Битовая карта, один бит на пиксель. Установленные биты рисуются текущим цветом; сброшенные прозрачны
`gfx_bitmap_file(x, y, w, h, name, name_length, offset)`	Битовая карта, читаемая хостом из файла приложения, начиная с `offset`
`gfx_qr(x, y, cell, text, length)`	QR-код до 200 байт. Возвращает число модулей на сторону. Клетка 0: только измерение
`void request_redraw(flags)`	Запрашивает перерисовку. Флаги: `HOST_REDRAW_NORMAL`, `_SOON`, `_CLEAN`, `_NO_BLINK`
`header_text(side, text, length)`	Задаёт текст приложения в заголовке хоста над его экраном: `HOST_HEADER_LEFT` или `_RIGHT`, до 63 байт. Длина 0 возвращает сторону хосту. Вызывается из любого обработчика

19 Kit

Вызов	Описание
`ui_text(x, y, style, text, length, max_width)`	Текст в стиле: `HOST_UI_TEXT`, `_TITLE`, `_CAPTION`, `_VALUE`; сокращается до `max_width` (0: без ограничения). Возвращает ширину
`ui_text_lines(x, y, style, text, length, width, max_lines)`	Текст с переносом по пробелам до `width`, не более `max_lines` строк (1 - 8); последняя строка сокращается с "...". Возвращает число строк

Вызов	Описание
`ui_text_width(style, text, length)`, `ui_text_height(style)`	Ширина текста и высота строки стиля
`ui_title(text, length)`	Заголовок страницы с линейкой. Возвращает y области содержимого
`ui_icon(x, y, icon, size)`	Встроенная иконка размера `HOST_UI_ICON_SMALL` (22 пикс.), `_MEDIUM` (44), `_LARGE` (99)
`ui_actions(ok, ok_length, back, back_length)`	Строка кнопок внизу, 44 пикселя (`HOST_UI_ACTIONS_HEIGHT`). Длина 0 убирает кнопку; единственная кнопка центрируется
`ui_button(x, y, name, name_length, action, action_length, mirrored)`	Одна кнопка: название на плашке и подпись действия рядом. Возвращает ширину
`ui_list_visible(list)`, `ui_list_first(list)`	Для `host_list_t` (count, selected или -1, top, height или 0 для оставшейся высоты, first): число видимых строк; индекс первой видимой строки
`ui_list_row(list, index, label, label_length, value, value_length, icon)`	Одна строка. Строки вне видимого диапазона пропускаются. Значение шириной до половины строки остаётся целым; подпись сокращается с "..."
`ui_notice(notice)`	Сообщение на всю поверхность по `host_notice_t`: first[64], second[64], detail[96], ok[32], back[32], icon, is_working. Пустые поля не выводятся
`ui_reason(status)`	Стандартное сообщение для статуса запроса. Возвращает 1, если нарисовано, 0 для статуса 2xx
`ui_value(x, y, font, value, value_length, unit, unit_length)`	Значение с единицей на базовой линии. Возвращает общую ширину
`ui_value_width(font, value, value_length, unit, unit_length)`	Ширина значения с единицей
`ui_cell(cell, picture)`	Ячейка по `host_cell_t` (x, y, width, height, name[48], value[48], size) с необязательной картинкой (0: нет). Размер картинки: 88 пикселей в ячейке на всю поверхность, иначе 44
`ui_cell_fit(cell, picture)`	Возвращает класс размера, подходящий ячейке; 1 - наибольший. Для всех ячеек экрана используйте наибольшее из возвращённых чисел
`ui_bar(x, y, width, value, maximum)`	Полоса высотой 14 пикселей: `value` из `maximum`
`ui_chart(chart, values)`	График из `host_chart_t` (x, y, width, height, kind, flags, decimals, count) и `count` чисел (1 - 128), от старых к новым. `kind`: `HOST_CHART_LINE`, `_AREA`, `_BARS`, `_STEPS`. `flags`: `HOST_CHART_AXIS` (линия основания), `_FROM_ZERO` (шкала включает 0), `_RANGE` (наибольшее и наименьшее значение подписями, с `decimals` знаками после запятой), `_MARK_LAST` (отметка на последнем значении). `HOST_CHART_NO_VALUE` - разрыв. Шкала охватывает наименьшее и наибольшее значение
`ui_frame(x, y, width, height, arrows)`	Рамка с необязательными стрелками: `HOST_UI_ARROW_UP`, `_DOWN`
`ui_picker(picker)`	Выбор значения по `host_picker_t`: above[64], chosen[64], below[64], ok[32], back[32]. Включает строку кнопок
`ui_date(date)`	Редактор даты и времени по `host_date_t`: year, month, day, hour, minute; has (`HOST_DATE_HAS_TIME | _DATE`); selected (`HOST_DATE_HOUR .. _YEAR` или `_NONE`); ok[32], back[32]

20 Навигация, хранение и сеть

Вызов	Описание
`void request_exit(void)`	Закрывает приложение: один шаг вниз по стеку. Допустим в обработчиках кнопок, BACK и таймера
`void request_focus(void)`	Запрашивает экран из фонового обработчика. Разрешается хостом, не чаще раза в 30 с
`notify(notice)`	Ставит в очередь уведомление по `host_notice_t`: first (обязательно), second, detail, icon. `HOST_ERR_NO_SPACE`, если в очереди уже четыре

Вызов	Описание
`header_status(side, icon, text, length)`	Задаёт иконку и до 31 байта текста в заголовке, показываются над любым экраном, пока приложение не на экране. Пустые иконка и текст освобождают сторону. Сбрасывается при перезагрузке
`timer_set(id, milliseconds)`	Однократный таймер, id 0 - 3. 0 мс отменяет. Новый вызов перезапускает
`alarm_set(id, seconds)`	Однократный будильник, id 0 - 3. 0 с отменяет
`alarm_set_at(id, int64_t unix_time)`	Однократный будильник по времени часов
`alarm_set_every(id, seconds, first_in)`	Повторяющийся будильник, период 60 - 86400 с. Первый вызов через `first_in` секунд (0: через период)
`kv_get(key, key_length, value, capacity)`	Читает значение. Возвращает его длину; `HOST_ERR_NOT_FOUND`; `HOST_ERR_NO_SPACE`, если буфер мал
`kv_set(key, key_length, value, length)`	Записывает значение. Длина 0 удаляет ключ. Для настроек типов `url` и `link` завершается ошибкой
`file_size(name, name_length)`	Возвращает размер файла или `HOST_ERR_NOT_FOUND`
`file_read(name, name_length, offset, bytes, capacity)`	Читает со смещения. Возвращает число прочитанных байт; 0 в конце файла и за ним
`file_write(name, name_length, offset, bytes, length)`	Записывает по смещению внутри файла или в его конец. Длина 0 по смещению 0 удаляет файл
`poll_now(poll)`	Немедленно выполняет опрос из манифеста
`poll_set_interval(poll, seconds)`	Задаёт интервал опроса, от 10 с до 1 суток. Сохраняется при перезагрузке
`http_request(tag, method, url, url_length, headers, headers_length, body, body_length)`	Ставит запрос в очередь. Методы: `HOST_HTTP_GET`, `_POST`, `_PUT`, `_DELETE`, `_PATCH`, `_HEAD`, `_PROPFIND`, `_REPORT`. Заголовки: строки вида "Name: value"
`data_value(index, text, capacity)`	Внутри `app_on_values()`: копирует значение пути `index` как текст без завершающего нуля. Возвращает его длину
`data_response(response)`	Внутри обработчика данных: заполняет `host_response_t`: status, length, retry_after, content_type[48], etag[64], last_modified[32], location[128], link[192]
`text_fill(pattern, pattern_length, arguments, count, text, capacity)`	Заменяет `{0}` .. `{3}` в шаблоне значениями `host_text_argument_t` (число либо текст и его длина). Возвращает длину
`void host_log(level, text, length)`	Записывает одну строку журнала до 200 символов. Уровни: `HOST_LOG_ERROR`, `_WARN`, `_INFO`, `_DEBUG`

21 Время

Вызов	Описание
`int64_t time_now(void)`	Время Unix в секундах; 0, пока время неизвестно
`time_zone_offset(void)`	Смещение местного времени от UTC в секундах, с учётом летнего времени
`time_local(parts), time_local_at(int64_t unix_time, parts)`	Заполняет `host_local_time_t`: year, month 1 - 12, day, weekday 0 = воскресенье, hour, minute, second
`int64_t time_from_local(parts)`	Время Unix для местных даты и времени. Дни за пределами месяца переходят в следующий
`time_text(int64_t unix_time, style, text, capacity)`	Форматирует момент времени на языке устройства. Стили: `HOST_TIME_CLOCK`, `_DATE`, `_WEEKDAY`, `_LONG_DATE`, `_NEAR`, `_NEAR_DAY`. Возвращает длину
`int64_t time_uptime_ms(void)`	Миллисекунды с запуска устройства; включая время сна
`uint32_t time_boot_id(void)`	Идентификатор текущей загрузки; меняется после каждой перезагрузки, никогда не равен 0. Сохраняйте его вместе с сохранённым временем работы

22 Системный API

Системный API - набор вызовов `sys_*`. Его используют собственные страницы хоста.

Системный API доступен любому приложению, в манифесте которого есть `"system": true`. Перед установкой такого приложения веб-страница предупреждает пользователя. Модуль, импортирующий вызов `sys_*` без этого объявления, отклоняется.

Вызов	Описание
`sys_info(what, text, capacity)`	Сведения об устройстве в виде текста: `SYS_INFO_VERSION`, `_NAME`, `_NETWORK`, `_ADDRESS`, `_WIFI` ("online", "connecting", "setup"), `_SLOT`, `_FREE_MEMORY`, `_TIME_ZONE`, `_JOIN_CODE`
`sys_number_get(what, number)`, `sys_number_set(what, value)`	Числовая настройка хоста с проверкой диапазона: `SYS_NUMBER_TIME_AUTOMATIC`, `_REFRESH_FULL_EVERY`, `_REFRESH_MIN_SECONDS`, `_REFRESH_FORCE_HOURS`, `_POWER_SAVING`, `_REFRESH_FAST`, `_FONT`, `_MENU_IDLE_MINUTES`, `_HEADER_CLOCK`
`sys_text_set(what, text, length)`	Текстовая настройка хоста: `SYS_TEXT_TIME_ZONE`, строка POSIX TZ
`sys_time_set(int64_t unix_seconds)`	Устанавливает часы. Завершается ошибкой, пока источник времени - сеть
`sys_app_list(ids, capacity)`, `sys_app_title(id, text, capacity)`, `sys_app_remove(id)`	Установленные приложения по id, до 16: список, название, удаление
`sys_app_default(void)`, `sys_app_set_default(id)`	id приложения по умолчанию; 0: нет
`sys_logo(x, y, scale)`, `sys_logo_width(scale)`, `sys_logo_height(scale)`	Словесный знак perINKpheraL квадратным шрифтом, масштаб 1 - 5: отрисовка, ширина, высота
`sys_do(action)`	`SYS_DO_RESTART`, `_FORGET_NETWORK`, `_FACTORY_RESET`. Требует экрана подтверждения в приложении

23 Библиотека SDK

`mkapp` компонует библиотеку с каждым приложением. Неиспользуемые функции удаляются. Заголовочные файлы находятся в `sdk/lib`.

`bool` - стандартный тип из `<stdbool.h>`. Текстовые аргументы завершаются нулём, если не указано иное.

app_text.h: короткие строки без printf

```
typedef struct { char *next; char *last; } app_text_t;      /* объявляется вызывающим кодом */

void    app_text_start(app_text_t *text, char *buffer, int32_t buffer_size);
void    app_text_add(app_text_t *text, const char *more);   /* лишний текст отбрасывается по границе символа */
void    app_text_add_number(app_text_t *text, int32_t value);               /* десятичное, "-" для отрицательных */
void    app_text_add_padded(app_text_t *text, int32_t value, int32_t digits);          /* 7, 2 -> "07" */
int32_t app_text_length(const char *text);                  /* длина любого текста, завершённого нулём */
void    app_text_copy(char *to, int32_t size, const char *from);                    /* всегда с завершающим нулём */
```

Буфер всегда завершается нулём и не выходит за свой размер. После каждого вызова текст в буфере полон.

app_draw.h, app_state.h: строки, состояние, настройки

```
#include "app_draw.h"                                    /* текст с завершающим нулём; внутри app_on_draw() */
int32_t app_text_width(int32_t font, const char *text);
int32_t app_draw_text(int32_t x, int32_t y, int32_t font, const char *text);        /* возвращает ширину */
void    app_draw_centered(int32_t center, int32_t y, int32_t font, const char *text);
void    app_draw_right(int32_t right, int32_t y, int32_t font, const char *text);

#include "app_state.h"                                   /* хранилище, ключ с завершающим нулём */
bool    app_state_load(const char *key, void *state, int32_t size, int32_t version);  /* первое поле: int32_t version */
void    app_state_save(const char *key, const void *state, int32_t size);             /* одна запись во флеш */
void    app_state_forget(const char *key);
int32_t app_setting(const char *key, char *text, int32_t text_size);                  /* длина; 0: не задано */
bool    app_setting_is(const char *key, const char *wanted);
bool    app_setting_is_on(const char *key);                                           /* toggle: true, кроме "0" */
bool    app_setting_number(const char *key, int32_t decimals, int32_t *number);
```

app_values.h: значения, извлечённые хостом

```
bool app_value_text(int32_t index, char *text, int32_t text_size);        /* false: значения нет или буфер мал */
bool app_value_number(int32_t index, int32_t decimals, int32_t *number);  /* "-3.46", 1 -> -35; false: значения нет */
bool app_parse_number(const char *text, int32_t decimals, int32_t *number);    /* то же для заданного текста */
```

Эти функции работают только внутри `app_on_values()`. `index` - позиция пути в списке `pick` манифеста.

app_ui.h: обработка кнопок на экранах Kit

```
typedef enum { APP_UI_IGNORED, APP_UI_CHANGED, APP_UI_DONE, APP_UI_CANCELLED } app_ui_result_t;

typedef host_list_t app_list_t;        /* count, selected (-1: нет), top, height, first */
app_ui_result_t app_list_button(app_list_t *list, int32_t event);    /* UP, DOWN: changed; OK: done; BACK: cancelled */
int32_t         app_list_first(app_list_t *list);                    /* вызывать в app_on_draw() перед отрисовкой строк */

typedef struct { int32_t count; int32_t chosen; bool wraps; } app_picker_t;
app_ui_result_t app_picker_button(app_picker_t *picker, int32_t event);
void            app_picker_draw(const app_picker_t *picker, const char *const *labels);

typedef struct { int32_t year, month, day, hour, minute, has, part, year_first, year_last; } app_date_t;
void            app_date_start(app_date_t *date, int32_t has, int32_t year_first, int32_t year_last);
app_ui_result_t app_date_button(app_date_t *date, int32_t event);    /* OK: следующая часть; done после последней части */
void            app_date_draw(const app_date_t *date);
int64_t         app_date_seconds(const app_date_t *date);            /* местное время; для UTC вычесть time_zone_offset() */
```

Чтобы обрабатывать BACK на этих экранах, экспортируйте `app_on_back()` и передавайте из него `BTN_BACK_PRESSED` в ту же функцию обработки кнопок.

app_alloc.h, json_pick.h и генерируемые файлы

```
void   *app_alloc(int32_t bytes);           /* NULL, если в арене мало места; память не обнуляется */
int32_t app_alloc_left(void);
int32_t app_alloc_mark(void);               /* app_alloc_release(mark) освобождает всё, выделенное после отметки */
void    app_alloc_release(int32_t mark);

void        json_pick_init(json_pick_t *pick, const char *const *paths, int path_count);    /* до 16 */
bool        json_pick_feed(json_pick_t *pick, const char *bytes, size_t length);      /* один фрагмент ответа */
void        json_pick_finish(json_pick_t *pick);
const char *json_pick_value(const json_pick_t *pick, int index);         /* NULL: значения нет */

#include "strings.h"      /* генерируется из strings/: enum { STR_NO_EVENTS, ... }; const char *app_string(int32_t id); */
#include "pictures.h"     /* генерируется из pictures/: static const uint8_t PICTURE_DROP[] = { width, height, bits }; */
```

`strings.h` и `pictures.h` генерируются для каждого приложения. Сборка ставит их первыми в пути поиска заголовочных файлов, поэтому системный заголовочный файл с тем же именем не используется.

`memcpy`, `memmove`, `memset` и `strlen` предоставляются, потому что компилятор генерирует их вызовы. Это вся библиотека C платформы. Они не объявлены ни в одном заголовке SDK: код, который вызывает их напрямую, объявляет прототип сам. `app_text_length()` из `app_text.h` заменяет `strlen`.

24 Лимиты

Константа	Значение	К чему относится
`HOST_API_VERSION`	2	`abi` в манифесте
`MANIFEST_MODULE_MAX_BYTES`	48 КБ	`app.wasm`
`MANIFEST_NAME_MAX / _TITLE_MAX`	13 / 23 символа	Имя и название приложения
`MANIFEST_POLLS_MAX / _HOSTS_MAX / _SETTINGS_MAX`	4 / 4 / 8	Опросы, адреса и настройки в манифесте
`MANIFEST_URL_MAX / _SETTING_VALUE_MAX`	511 / 255 байт	URL опроса; значение настройки
`HOST_TEXT_MAX`	127 байт	Текст на один вызов отрисовки или измерения
`HOST_HEADER_TEXT_MAX / HOST_STATUS_TEXT_MAX`	63 / 31 байт	Текст в заголовке; текст статуса в заголовке
`HOST_KV_KEY_MAX / _VALUE_MAX / _TOTAL_MAX`	15 / 1024 / 4096	Ключ хранилища; значение; всего на приложение
`HOST_FILE_NAME_MAX / HOST_FILES_MAX`	23 / 64 КБ	Имя файла; общий размер файлов на приложение
`HOST_DATA_CHUNK_MAX`	1024 байта	Один фрагмент ответа
`HOST_ANSWER_MAX / HOST_ANSWER_PIECES_MAX`	256 КБ / 4 МБ	Ответ с извлечёнными значениями; потоковый ответ
`HOST_VALUE_MAX`	63 байта	Одно извлечённое значение
`HOST_HTTP_HEADERS_MAX / _BODY_MAX / _PENDING_MAX`	1023 / 512 / 2	Заголовки запроса; тело; запросов в очереди на устройство
`HOST_REQUEST_FIRST .. _LAST`	16 .. 255	Теги для запросов из кода
`HOST_TIMER_COUNT / HOST_ALARM_COUNT`	4 / 4	Таймеры и будильники на приложение
`HOST_ALARM_EVERY_MIN .. _MAX`	60 .. 86400 с	Период повторяющегося будильника
`HOST_TEXT_ARGUMENTS_MAX`	4	Аргументы `text_fill()`
`HOST_QR_TEXT_MAX / HOST_LOG_TEXT_MAX`	200 / 200	Текст QR-кода; строка журнала
`HOST_ICON_CELLS / HOST_ICON_CELL_MAX`	11 / 16	Сетка иконки; наибольший размер клетки в пикселях
`Таблица строк одного языка`	8 КБ	Загружается хостом

`sdk/host_api.h` - основной контракт. Если этот документ и заголовочный файл расходятся, верен заголовочный файл.